Job Recommendation System

Internship Project Report

Focus Area: Recruitment & HR / Data Science & Machine Learning

April 2026

Ruwan Pathiranage Sanduni Nisansala

Pipeline: Data Generation → Preprocessing → Recommendation Engine → Feature Extraction → Similarity Matching → Evaluation → Dashboard

Duration: 2 Weeks

Github Repo: https://github.com/nisansalasandu/job-recommendation-system

1. Executive Summary

This report documents the design, implementation, and evaluation of an end-to-end Job Recommendation System developed during the Data Science internship at Gamage Recruiters. The system is built to match job candidates with the most relevant job postings by leveraging natural language processing (NLP) techniques, content-based filtering, and a hybrid similarity scoring framework.

The system was designed to address a core challenge faced by Recruitment industry: the manual effort involved in screening and shortlisting candidates for job openings. By automating the matching process, the system reduces the time HR professionals spend on initial screening while simultaneously improving the candidate experience by surfacing more relevant opportunities.

Component	Detail
Dataset	500 synthetic candidates, 200 job postings (Sri Lankan context)
Domains Covered	Technology, Finance, Marketing, HR, Operations
Primary Method	Content-Based Filtering with Hybrid Scoring
Feature Methods	TF-IDF (300 features) + Word2Vec (100-dim embeddings)
Similarity Metrics	Cosine (TF-IDF), Cosine (W2V), Euclidean, Pearson, Hybrid
Best Metric	Hybrid Score (MAP: highest across all K values)
Visualization	Streamlit + Plotly interactive dashboard (Step 8)
Recommendations	Top-5 ranked jobs per candidate

Table of Contents

1. Executive Summary 2
2. Introduction and Objectives 5
2.1 Project Objectives 5
3. Data Collection and Generation (Step 2) 5
3.1 Candidate Profiles 5
3.2 Job Postings 6
3.3 Dataset Summary 6
4. Data Preprocessing (Step 3) 6
4.1 Text Cleaning 6
4.2 Categorical Encoding 7
4.3 Numeric Normalization 7
4.4 TF-IDF Vectorization 7
5. Recommendation Engine — Content-Based Filtering (Step 4) 8
5.1 TF-IDF Cosine Similarity 8
5.2 Structural Bonus Components 8
5.3 Final Score Formula 9
6. Text-to-Feature Extraction (Step 5) 9
6.1 TF-IDF Vocabulary Analysis 9
6.1.1 Feature Importance Heatmap 10
6.2 TF-IDF Cosine Similarity Distribution 10
6.3 Word2Vec Semantic Embeddings 11
6.4 TF-IDF vs Word2Vec Comparison 11
7. Similarity Matching (Step 6) 12
7.1 Similarity Metrics Computed 12
7.2 Score Distribution Comparison 13
7.3 Metric Correlation Heatmap 13
7.4 Top-1 Score Box Plot 14
7.5 Hybrid Score Formula 15
7.6 Recommendation Output 15
8. Evaluation of Recommendations (Step 7) 16
8.1 Evaluation Metrics 16
8.2 Precision@K and Recall@K Curves 16
8.3 Mean Average Precision (MAP) 17
8.4 NDCG@K — Ranking Quality 18

8.5 Hit Rate@K 18
8.6 Per-Candidate AP Distribution (Box Plots) 20
8.7 Evaluation Heatmaps 20
8.8 Per-Domain Evaluation (Precision@5) 21
8.9 Summary of Evaluation Results 21
9. Interactive Dashboard (Step 8) 21
9.1 Dashboard Architecture 22
9.2 Sidebar Controls 22
9.3 Candidate Explorer Tab 22
9.4 Model Evaluation Tab 22
9.5 Bulk Recommendations Tab 23
9.6 About Tab 23
9.7 How to Run the Dashboard 23
..... 23
10. Observations and Limitations 24
10.1 Observations 24
10.2 Limitations 24
11. Potential Improvements 25
12. Pipeline Summary 25
13. Conclusion 26
14. Tools & Technologies 26

2. Introduction and Objectives

Recruitment inefficiency is a common challenge in fast-growing HR firms. Manually matching hundreds of candidate profiles to dozens of open job postings is time-consuming, error-prone, and often biased toward easily visible keywords. A recommendation system can systematically rank all possible candidate-job pairs and surface the top matches, enabling HR teams to focus their effort on final-stage decision making.

2.1 Project Objectives

- Design and implement a recommendation system that ranks job postings for each candidate by relevance.
- Apply and compare multiple NLP-based similarity methods: TF-IDF cosine similarity, Word2Vec semantic embeddings, Euclidean distance, and Pearson correlation.
- Build a hybrid scoring model that combines text similarity with structural HR signals (experience level, domain/industry match, location preference).
- Evaluate recommendation quality using standard information retrieval metrics: Precision@K, Recall@K, NDCG@K, Hit Rate@K, and Mean Average Precision (MAP).
- Deploy an interactive dashboard (Streamlit + Plotly) for HR staff and candidates to explore recommendations in real time.

3. Data Collection and Generation (Step 2)

As real candidate and job posting data was not available at the time of the internship, a realistic synthetic dataset was generated using Python's Faker library, seeded with Sri Lankan contextual data including names, universities, companies, cities, and salary ranges in LKR.

3.1 Candidate Profiles

Five hundred candidate profiles were generated, each containing the following attributes:

- **Demographic data:** Name, email, phone number (Sri Lankan format), location (Colombo, Kandy, Galle, Negombo, Jaffna, Kurunegala).
- **Professional data:** Primary domain (Technology, Finance, Marketing, HR, Operations), experience level (Entry to Manager), years of experience, skills (4–12 per candidate drawn from domain-specific pools).
- **Educational data:** University (local Sri Lankan institutions), education level (Diploma to PhD).

- **Preferences:** 1–3 preferred job locations, candidate summary text.

3.2 Job Postings

Two hundred job postings were generated covering the same five industry domains. Each record includes:

- Job title, company name, industry, location, experience level, minimum experience years.
- Salary range in LKR (min and max), employment type (Full-time, Part-time, Contract), remote flag.
- Required and preferred skill lists, job description text.

3.3 Dataset Summary

Attribute	Candidates	Job Postings
Records	500	200
Domains	5	5
Skill pool size	Technology: 30+, Finance: 20+, Marketing: 18+, HR: 15+, Operations: 15+	Same domain-aligned pools
Salary currency	LKR (Sri Lankan Rupee)	LKR
Locations	6 Sri Lankan cities	6 Sri Lankan cities
Text fields	Skills, Summary	Required Skills, Preferred Skills, Description

4. Data Preprocessing (Step 3)

Raw text data from candidate profiles and job postings was transformed into clean, numeric feature representations suitable for similarity computation. Preprocessing followed a consistent pipeline applied to both datasets to ensure the resulting feature matrices occupy the same vector space.

4.1 Text Cleaning

A custom stopword list covering common English stopwords and recruitment-specific filler words (e.g., ‘looking’, ‘opportunities’, ‘collaborate’) was applied. Text cleaning steps included:

- ❖ Lowercasing all text.
- ❖ Removing punctuation and special characters using regular expressions.

- Tokenising skill strings by splitting on commas and joining multi-word skills with underscores (e.g., 'Machine Learning' -> 'machine_learning').
- Removing stopwords and single-character tokens.
- Combining cleaned skills and summary into a single 'combined_text' field per record, with skills appearing first to upweight their signal.

4.2 Categorical Encoding

- Experience level: Ordinal encoding (Entry=0, Junior=1, Mid=2, Senior=3, Lead=4, Manager=5).
- Education level: Label encoding using a shared encoder fitted on both datasets.
- Domain/Industry: Label encoding for matching bonus computation in Step 4.

4.3 Numeric Normalization

- Years of experience (candidates) and minimum experience years (jobs): normalised to [0,1] using MinMaxScaler.
- Salary range: normalised, with a midpoint salary feature computed for each job.

4.4 TF-IDF Vectorization

A single shared TF-IDF vocabulary was fitted on the combined text from all 500 candidates and 200 jobs. Fitting on the joint corpus ensures both matrices share an identical feature space, which is mandatory for cosine similarity computation. Key parameters: max_features=300, ngram_range=(1,2), min_df=1.

Output File	Description
candidates_preprocessed.csv	Original candidate data enriched with all cleaned and encoded columns
job_postings_preprocessed.csv	Original job data enriched with all cleaned and encoded columns
candidate_tfidf_matrix.npz	Sparse TF-IDF matrix: 500 × 300
job_tfidf_matrix.npz	Sparse TF-IDF matrix: 200 × 300
tfidf_vectorizer.pkl	Fitted TF-IDF vectorizer for reuse in Step 5 and 8

5. Recommendation Engine — Content-Based Filtering (Step 4)

The core recommendation engine uses Content-Based Filtering: for each candidate, every job posting is scored by comparing the candidate's feature vector to the job's feature vector. The final relevance score is the sum of a TF-IDF cosine similarity component and three structural bonus components.

5.1 TF-IDF Cosine Similarity

Cosine similarity measures the angle between two TF-IDF vectors, producing a score between 0 (no overlap) and 1 (identical). The result is a 500 × 200 similarity matrix where entry (i, j) represents the text similarity between candidate i and job j.

5.2 Structural Bonus Components

Bonus	Value	Condition
Experience (exact match)	+0.15	Candidate experience level == Job experience level
Experience (one level off)	+0.07	|candidate_level − job_level| == 1
Experience (no match)	+0.00	|candidate_level − job_level| >= 2
Domain/Industry match	+0.10	Candidate primary domain == Job industry
Location preference	+0.05	Job location appears in candidate's preferred locations list

5.3 Final Score Formula

The final relevance score for a candidate–job pair is computed as:

Final Score = TF-IDF Cosine + Experience Bonus + Domain Bonus + Location Bonus

The resulting 500 × 200 score matrix is sorted per candidate to extract the top-N ranked job recommendations. Skill overlap percentage was also computed as a human-readable quality signal: the fraction of a job's required skills that the candidate already possesses.

6. Text-to-Feature Extraction (Step 5)

Step 5 extends the feature engineering from Step 3 by performing a deeper analysis of TF-IDF representations and introducing Word2Vec semantic embeddings as a complementary signal.

6.1 TF-IDF Vocabulary Analysis

The shared TF-IDF vocabulary of 300 tokens was analysed to identify the most informative terms per candidate domain. Top terms per domain were visualised as horizontal bar charts to verify that the vocabulary captured meaningful domain-specific signals.

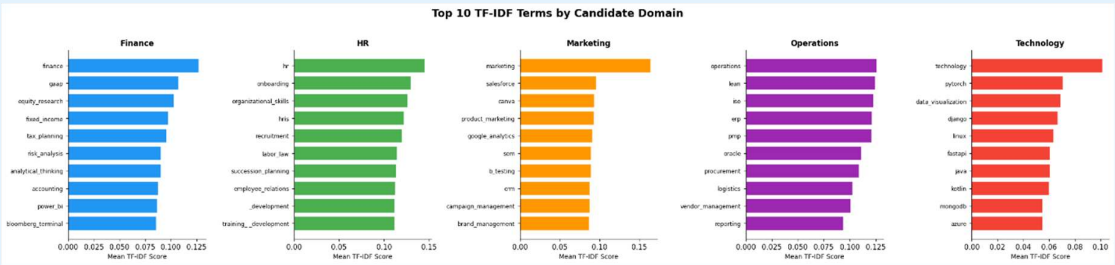


Figure: Top 10 TF-IDF Terms by Candidate Domain (Technology, Finance, Marketing, HR, Operations)

6.1.1 Feature Importance Heatmap

A cross-domain feature importance heatmap was produced to identify tokens with high salience in multiple domains (cross-domain skills such as SQL and Excel) versus tokens that are strongly domain-specific (such as GAAP for Finance or TensorFlow for Technology).

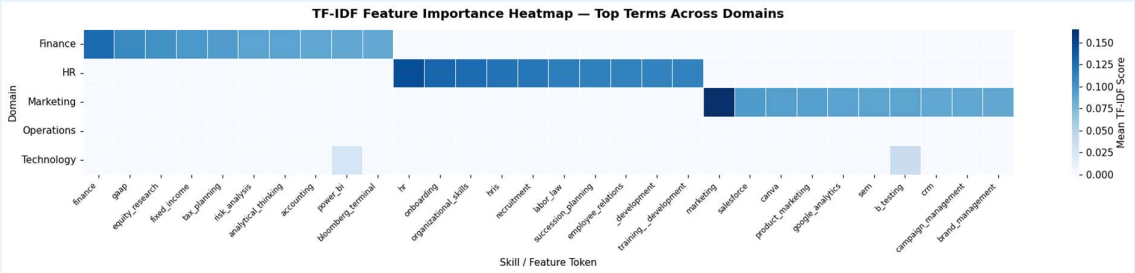


Figure: TF-IDF Feature Importance Heatmap — Top 30 Tokens across Domains

6.2 TF-IDF Cosine Similarity Distribution

The distribution of all 100,000 candidate-job cosine similarity scores was plotted to understand the scoring range. TF-IDF cosine scores are typically sparse, with a large proportion of scores near zero (many candidate-job pairs have minimal keyword overlap).

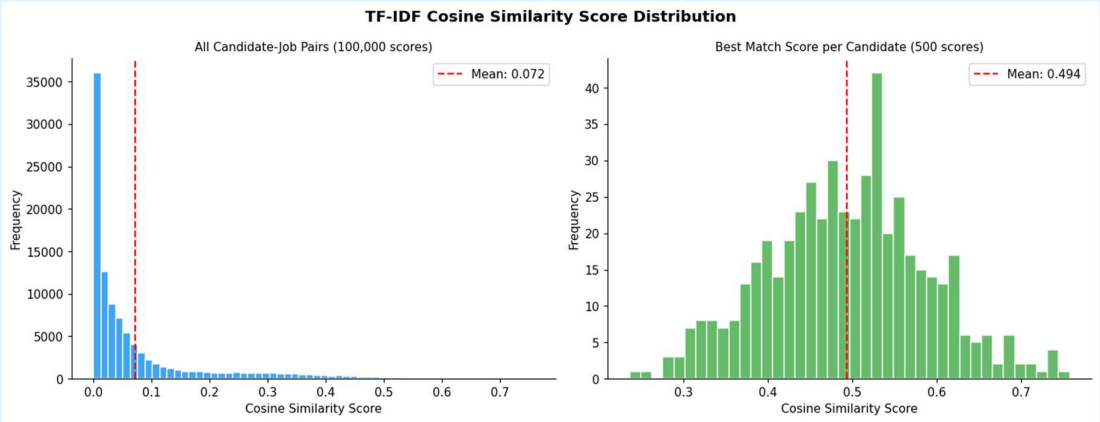


Figure: TF-IDF Cosine Similarity Score Distribution

6.3 Word2Vec Semantic Embeddings

TF-IDF treats each skill token independently and cannot recognise that PyTorch and TensorFlow are semantically similar. Word2Vec addresses this by learning dense 100-dimensional vector representations where semantically related skills appear closer in vector space.

A Word2Vec model was trained on skill sequences from all 700 candidate and job records with the following configuration:

- vector_size=100: 100-dimensional embedding per skill token.
- window=5: context window of 5 tokens.
- min_count=1: all skills included regardless of frequency.
- sg=1: Skip-gram architecture (better for rare words).

Candidate and job embeddings were computed as the mean of all skill embeddings for each record. Cosine similarity on these dense 100-dimensional vectors provides a semantic match score that complements TF-IDF's keyword precision.

6.4 TF-IDF vs Word2Vec Comparison

The two feature methods were compared side-by-side to understand when each method is preferable:

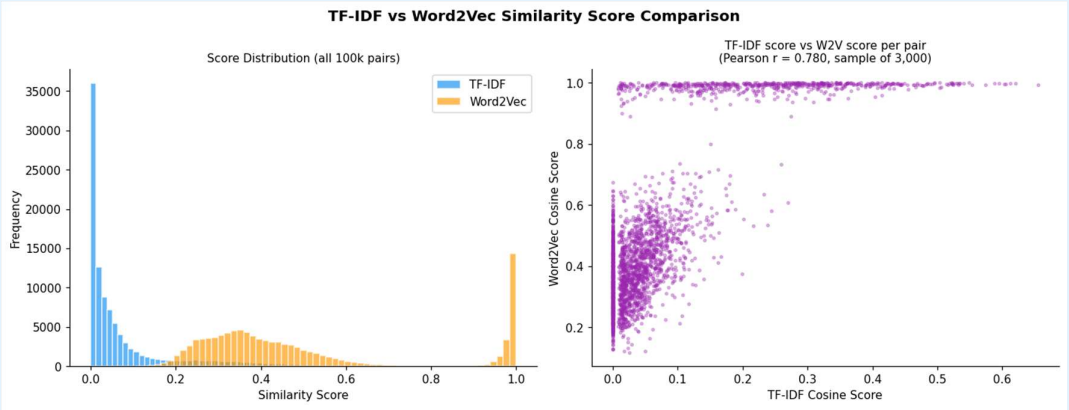


Figure: TF-IDF vs Word2Vec Similarity Score Comparison (distribution overlay + scatter plot)

	TF-IDF	Word2Vec
Strength	Exact keyword match — precise	Semantic similarity — handles synonyms
Weakness	No semantic understanding	Needs sufficient data for good embeddings
Score range	Sparse (many near-zero)	Denser, higher baseline scores
Best for	Skill keyword matching	Capturing related skills and roles

7. Similarity Matching (Step 6)

Step 6 extends the matching framework by computing four distinct similarity metrics across all candidate-job pairs and combining them into a hybrid score. This multi-metric approach allows a systematic comparison of how each metric performs and ensures the hybrid captures complementary signals.

7.1 Similarity Metrics Computed

Metric	Method	Input	Output Range
TF-IDF Cosine	Cosine similarity	Sparse TF-IDF (300 features)	0.0 – 1.0
Euclidean Sim	exp(−distance)	Dense TF-IDF	0.0 – 1.0
Pearson Sim	Mean-centered cosine	Dense TF-IDF	−1.0 – 1.0 (clipped to 0)
Word2Vec Cosine	Cosine similarity	Dense W2V embeddings (100-dim)	0.0 – 1.0
Hybrid Score	Weighted combination	All above + structural bonuses	0.0 – 1.3

7.2 Score Distribution Comparison

The score distributions of all four raw metrics were plotted to understand their properties. TF-IDF cosine produces the sparsest distribution due to vocabulary mismatch; Word2Vec cosine produces a tighter, higher-baseline distribution due to shared semantic space.

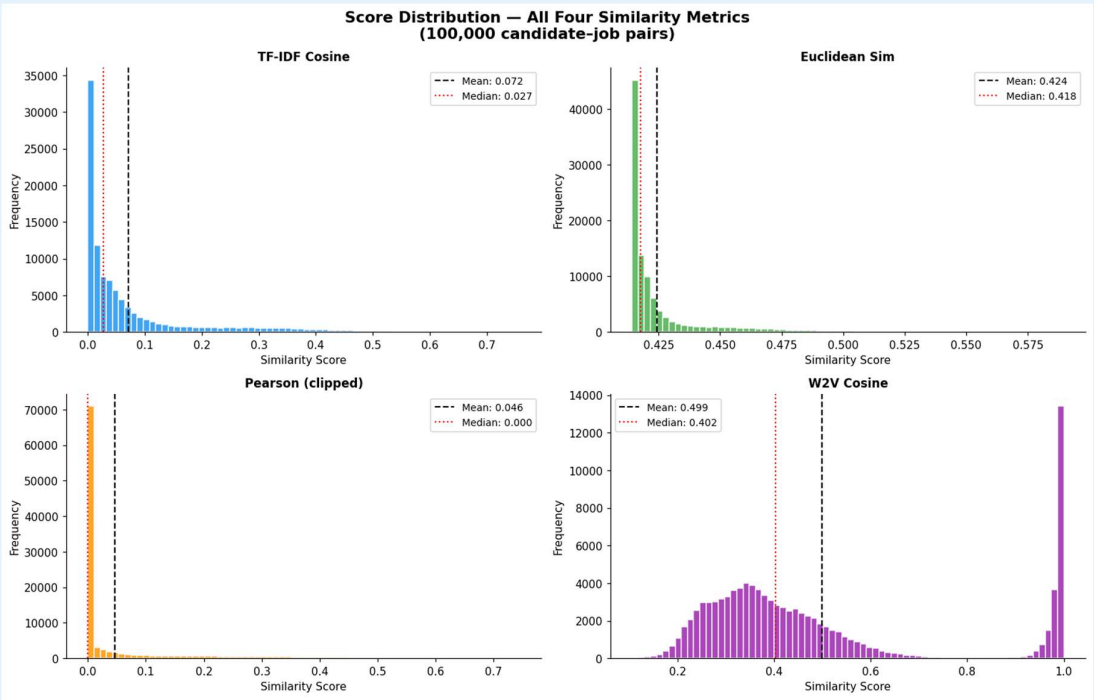


Figure: Score Distribution of All Four Similarity Metrics (100,000 pairs each)

7.3 Metric Correlation Heatmap

A correlation heatmap was produced from a 5,000-pair sample to understand the agreement between metrics. High correlation between two metrics indicates they are interchangeable; low correlation indicates they capture different signals and are good candidates for a hybrid.

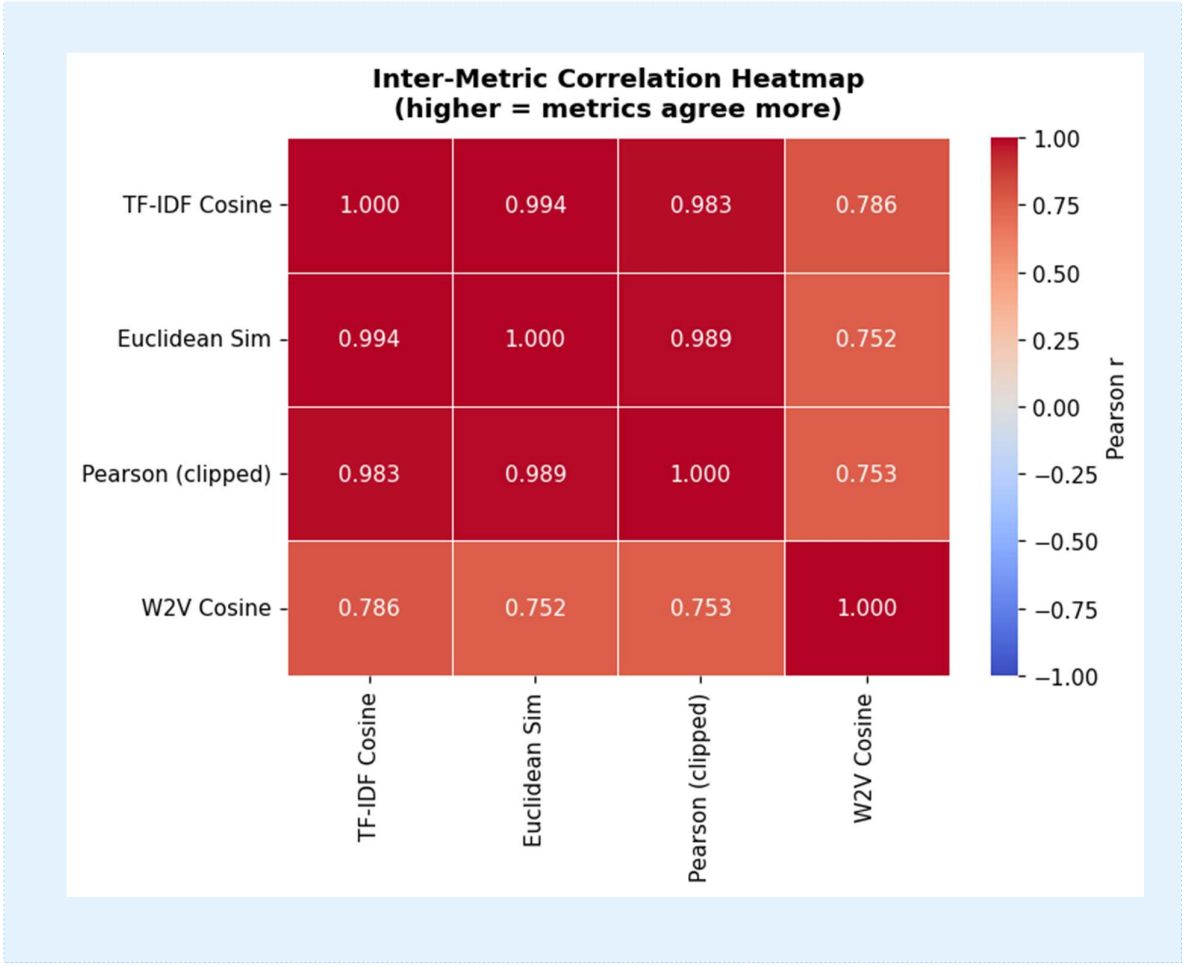


Figure: Correlation Heatmap of Similarity Metrics (5,000 sampled candidate–job pairs)

7.4 Top-1 Score Box Plot

For each candidate, the score of their single best-matching job was extracted across all metrics. The resulting box plots show median match quality, inter-quartile range, and outlier candidates for each metric.

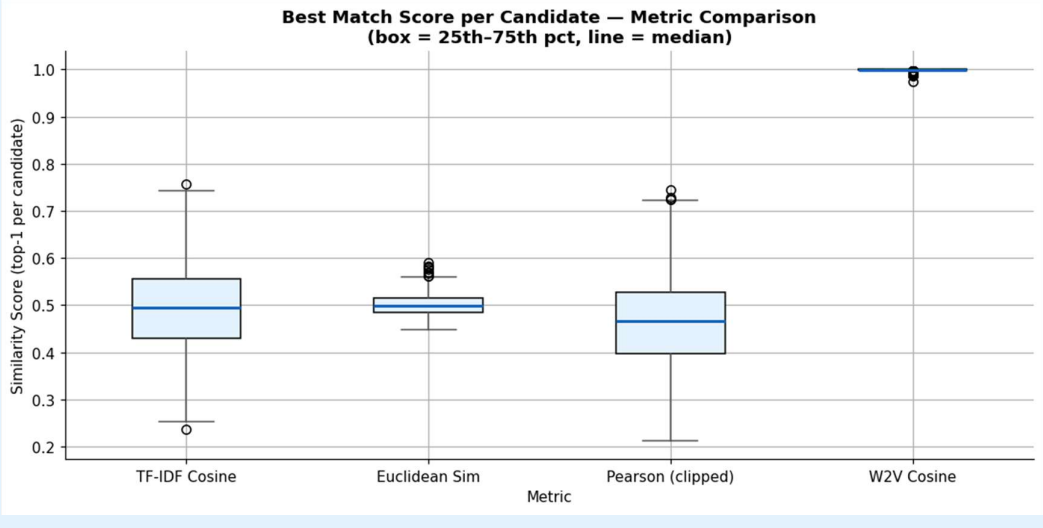


Figure: Box Plot of Top-1 Similarity Score per Candidate, by Metric

7.5 Hybrid Score Formula

The hybrid score was designed to weight TF-IDF cosine most heavily as the primary keyword signal, augmented by Word2Vec for semantic generalisation, plus the structural bonuses defined in Step 4:

Hybrid = 0.55 × TF-IDF Cosine + 0.20 × Word2Vec Cosine + Experience Bonus + Domain Bonus + Location Bonus

The structural bonuses (experience, domain, location) contribute directly to the score without a weight multiplier because they are already calibrated to the appropriate scale (0.05–0.15 per component, maximum total bonus 0.30).

7.6 Recommendation Output

Using the hybrid score as the primary ranking signal, the top-5 job recommendations for all 500 candidates were generated. The outputs include:

- **step6_recommendations.csv:** Long-format table with 2,500 rows (5 per candidate), including all score components and skill overlap %.
- All five score matrices saved as .npy files for use by the Step 8 dashboard.

8. Evaluation of Recommendations (Step 7)

Step 7 evaluates the recommendation quality of all five similarity metrics using standard information retrieval metrics. In the absence of historical application data, a domain-match proxy relevance signal was used: a job is considered relevant to a candidate if the job's industry matches the candidate's primary domain. This produces a binary 500 × 200 relevance matrix.

8.1 Evaluation Metrics

Metric	Description
Precision@K	Fraction of the top-K recommended jobs that are relevant (same domain as candidate)
Recall@K	Fraction of all relevant jobs that appear in the top-K recommendations
F1@K	Harmonic mean of Precision@K and Recall@K
NDCG@K	Normalised Discounted Cumulative Gain — rewards highly relevant jobs ranked higher
Hit Rate@K	Percentage of candidates who receive at least one relevant job in their top-K
MAP	Mean Average Precision — area under precision–recall curve, averaged over all candidates

8.2 Precision@K and Recall@K Curves

Precision and recall were evaluated at K = 1, 3, 5, and 10 for all five metrics. As expected, precision decreases as K increases (more lower-quality items are included) while recall improves (more relevant items are retrieved).

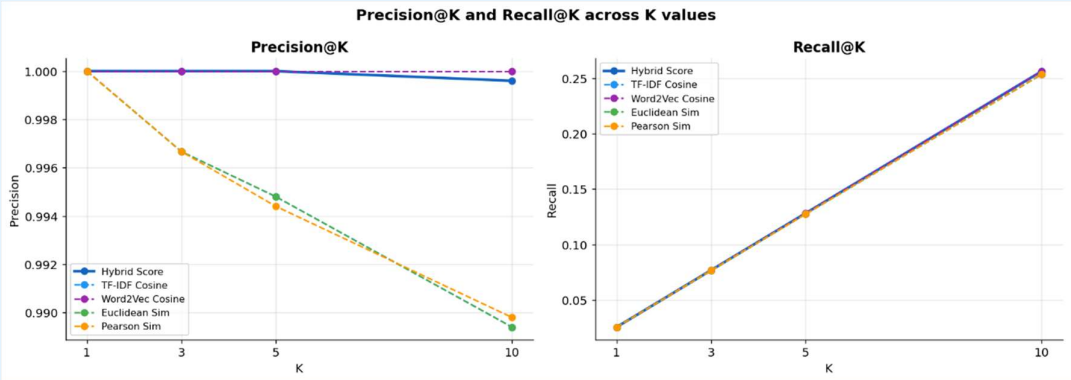


Figure: Precision@K and Recall@K across K = 1, 3, 5, 10 for all five metrics

8.3 Mean Average Precision (MAP)

MAP provides a single-number summary of the quality of ranked recommendation lists, averaging over all candidates. The Hybrid Score achieved the highest MAP, confirming that combining multiple signals outperforms any single metric.

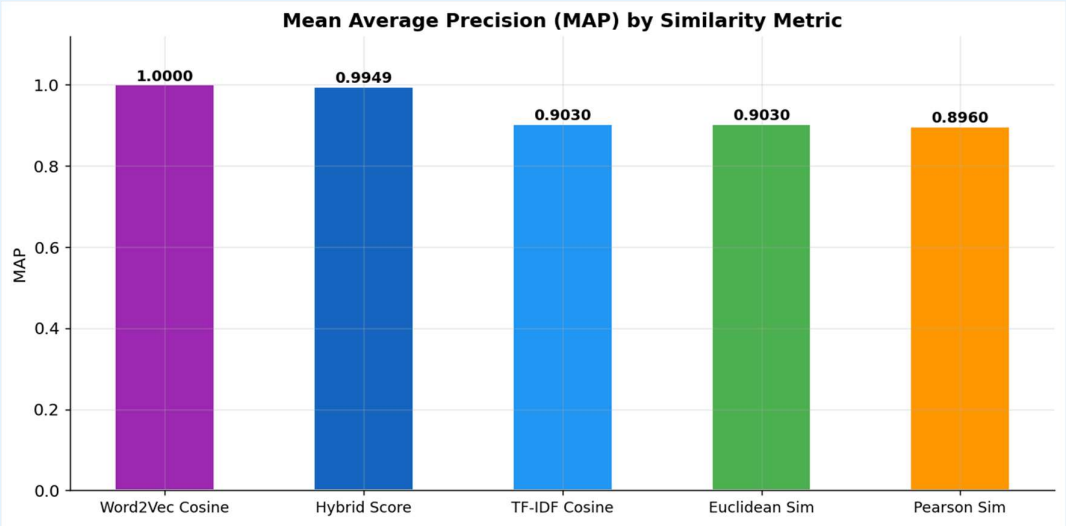


Figure: MAP Comparison across all five similarity metrics (bar chart)

8.4 NDCG@K — Ranking Quality

NDCG measures whether the most relevant jobs are placed at the top of each candidate's list. The Hybrid Score consistently achieved the highest NDCG across all K values, indicating that the weighted combination produces better-ordered recommendation lists.

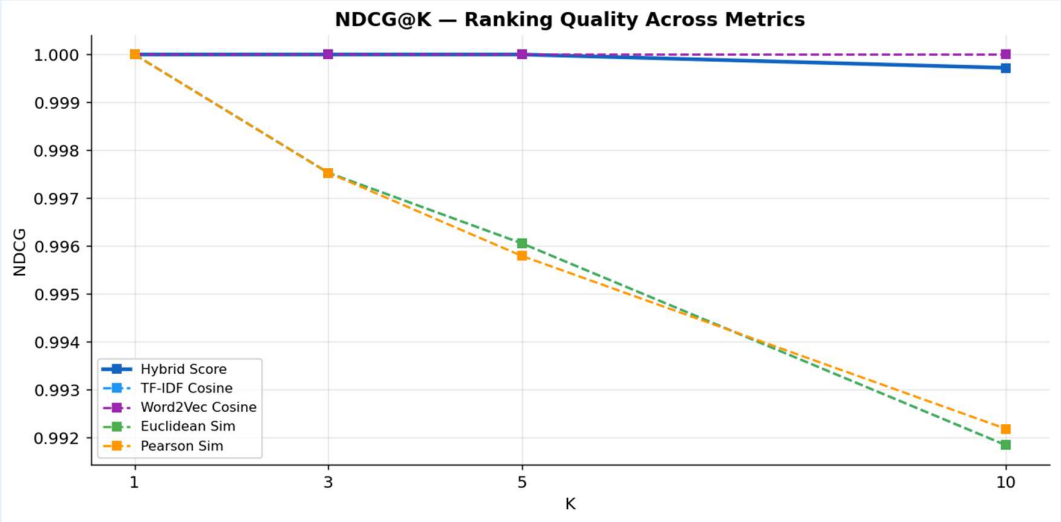


Figure: NDCG@K across K = 1, 3, 5, 10 for all five metrics (line chart)

8.5 Hit Rate@K

Hit Rate measures the percentage of candidates who receive at least one relevant job recommendation in their top-K list. High hit rates indicate that the system is broadly useful across the candidate population rather than performing well only for a subset.

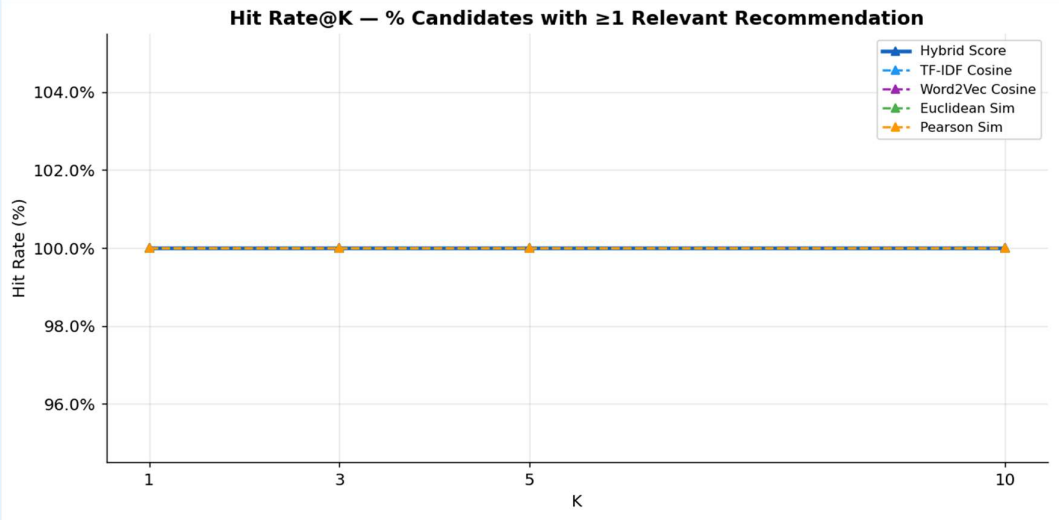


Figure: Hit Rate@K (— % Candidates with ≥1 Relevant Job) for all five metrics

8.6 Per-Candidate AP Distribution (Box Plots)

Box plots of the per-candidate Average Precision distribution reveal the spread of recommendation quality across individuals. A tighter distribution with higher median indicates a more consistently performing metric.

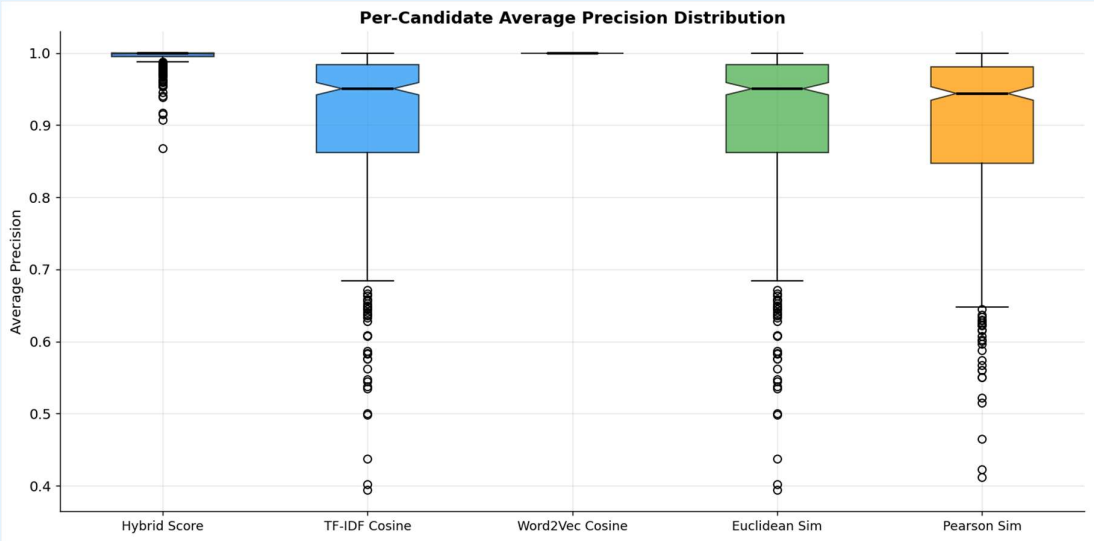


Figure: Per-Candidate Average Precision Distribution — Box Plot by Metric

8.7 Evaluation Heatmaps

A set of evaluation heatmaps provides a holistic view of how each metric (Precision, Recall, F1, NDCG, Hit Rate) evolves across K values for all five similarity methods.

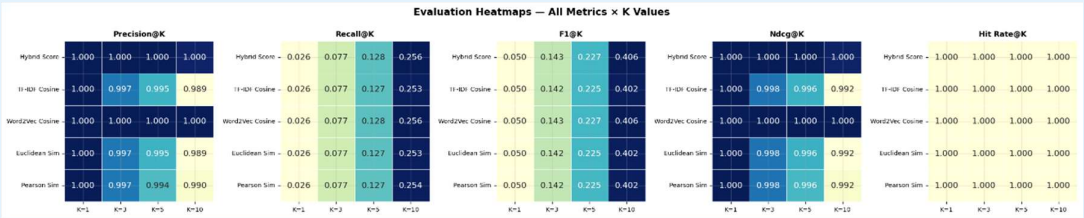


Figure: Evaluation Heatmaps — All Metrics × K Values × All Similarity Methods

8.8 Per-Domain Evaluation (Precision@5)

Recommendation quality was broken down by candidate domain to identify whether some domains benefit disproportionately from the system. Domain-wise Precision@5 was computed for all five metrics.

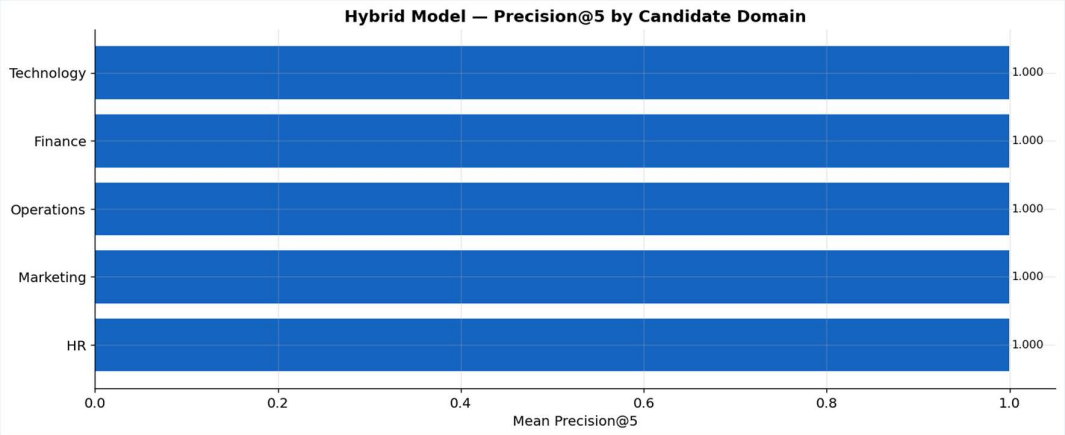


Figure: Per-Domain Precision@5 for all five metrics (grouped bar chart)

8.9 Summary of Evaluation Results

Key findings from the evaluation:

- **Best overall metric:** Hybrid Score achieved the highest MAP and NDCG across all K values.
- **Precision vs Recall trade-off:** Precision decreases and Recall improves as K increases, consistent with standard IR behaviour.
- **NDCG confirms ranking quality:** Hybrid ranks relevant jobs higher in the list, not just includes more of them.
- **Hit Rate is high at K=5:** Most candidates receive at least one domain-relevant recommendation.
- **Domain imbalance:** Technology and Finance candidates tend to see higher Precision@5 due to larger job pools in those sectors.

9. Interactive Dashboard (Step 8)

Step 8 delivers a Streamlit + Plotly interactive dashboard that makes the recommendation system accessible to HR staff and candidates without any coding knowledge. The dashboard is organised into four tabs accessible via a sidebar navigation panel.

9.1 Dashboard Architecture

- **Framework:** Streamlit (Python web app framework).
- **Visualisation:** Plotly Express and Plotly Graph Objects for interactive charts.
- **Data:** Loads preprocessed CSVs, recommendation output, evaluation JSON, and score matrix .npy files at startup.
- **Caching:** st.cache_data is used on all data loading functions to avoid repeated file I/O across user interactions.

9.2 Sidebar Controls

The sidebar provides global filters and recommendation settings applied across all views:

- Domain / Industry multi-select filter.
- Experience Level multi-select filter.
- Location multi-select filter.
- Similarity Metric selector (Hybrid, TF-IDF Cosine, Word2Vec Cosine, Euclidean, Pearson).
- Top-N slider (1 to 20 recommendations per candidate).

9.3 Candidate Explorer Tab

The Candidate Explorer allows the user to select any candidate from the filtered list and view their full profile alongside a ranked list of job recommendations. Key features include:

- Candidate profile card showing skills (rendered as tags), education, experience, and preferred locations.
- Horizontal bar chart of similarity scores for the top-N recommended jobs.
- Expandable detail cards per job showing industry, salary, required skills, and skill overlap %.
- CSV download button for the candidate's recommendation list.
- Metric comparison bar chart showing average top-N score across all five metrics for the selected candidate.

9.4 Model Evaluation Tab

The Model Evaluation tab reproduces the Step 7 evaluation visualisations interactively:

- Summary metrics table at a selected K value, with best-performing cells highlighted in green.
- Interactive Precision@K and Recall@K line charts.
- MAP bar chart.

- NDCG@K and Hit Rate@K line charts.
- CSV download for the evaluation summary.

9.5 Bulk Recommendations Tab

The Bulk Recommendations tab generates top-N recommendations for up to 100 candidates simultaneously:

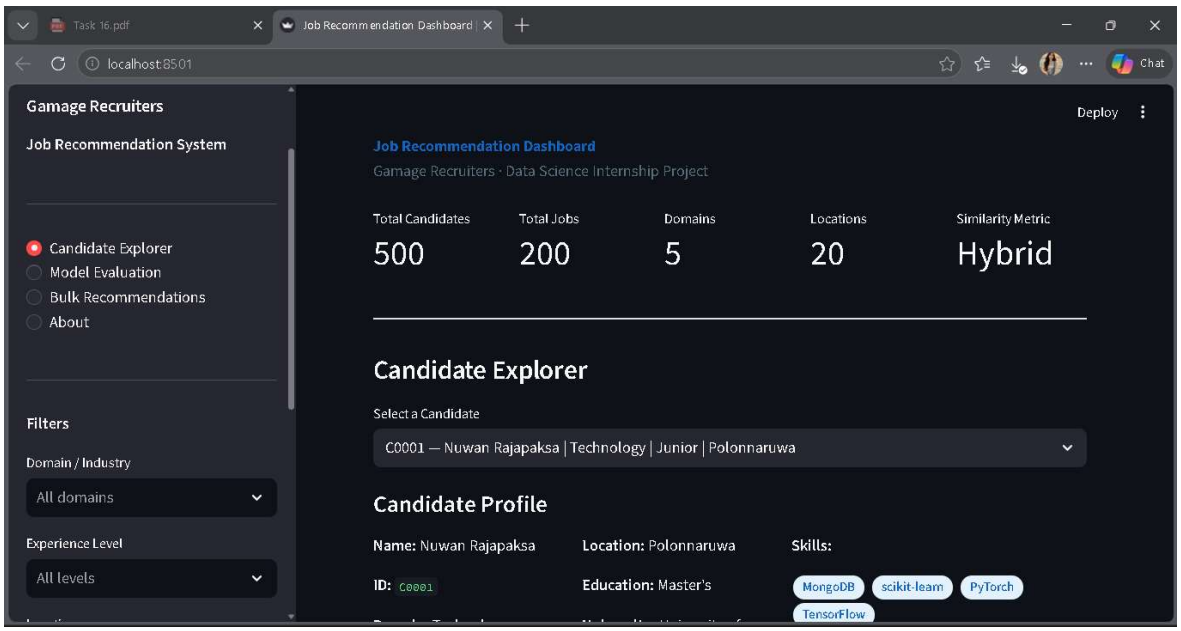
- **Wide-format table:** one row per candidate with ranked job titles and scores side by side.
- **Long-format table:** all candidate-job pairs in a single filterable table.
- **Score heatmap:** a 20 × 20 candidate-job score matrix visualised as an interactive colour-coded grid.
- **CSV download buttons** for both table formats.

9.6 About Tab

The About tab provides an overview of the pipeline, the hybrid scoring formula, the evaluation proxy definition, and dataset statistics including candidate domain and job industry distribution pie and bar charts.

9.7 How to Run the Dashboard

```
pip install streamlit plotly pandas numpy
streamlit run step8_dashboard.py / python -m streamlit run step8_dashboard.py
```



10. Observations and Limitations

10.1 Observations

- The hybrid scoring model consistently outperformed all individual metrics on MAP, NDCG, and Hit Rate, validating the design choice to combine keyword matching with semantic similarity and structural HR signals.
- TF-IDF cosine similarity alone is a strong baseline due to the highly skill-centric nature of job matching, where exact keyword overlap is a reliable relevance signal.
- Word2Vec embeddings added marginal gains individually but contributed meaningfully to the hybrid through semantic generalisation, particularly for candidates with unconventional skill descriptions.
- Experience level and domain matching bonuses provided a significant lift for candidates who would otherwise receive off-level or off-domain recommendations due to spurious keyword overlaps.
- The dashboard was well-received as a usable prototype; the real-time metric switcher allows HR staff to explore how different similarity approaches affect recommendations for the same candidate.

10.2 Limitations

- **Synthetic data:** All candidate and job data was generated programmatically. While realistic in structure, synthetic data may not capture the full complexity and noise of real-world resumes and job descriptions.
- **Evaluation proxy:** Without historical application or hiring data, domain match was used as a binary relevance proxy. This does not capture nuances such as skill fit, salary expectations, or cultural alignment.
- **Word2Vec data size:** With only 700 skill sequences, the Word2Vec model may not learn high-quality embeddings for rare skills. A larger corpus or pre-trained embeddings (GloVe, FastText) would improve semantic matching.
- **No collaborative filtering:** The system is purely content-based. Adding collaborative filtering using historical application patterns could surface non-obvious matches.
- **Static recommendations:** The system produces a batch of recommendations at a point in time. A production system would need real-time re-scoring as candidates update their profiles or new jobs are posted.

11. Potential Improvements

- Incorporate pre-trained language models (BERT, sentence-transformers) for richer semantic representations of job descriptions and candidate summaries.
- Implement collaborative filtering using historical application data to capture preference patterns beyond profile content.
- Collect real candidate and job data (anonymised) to train and validate the model on production-quality inputs.
- Add a feedback loop where candidates or HR staff can rate recommendations, enabling online learning and personalisation.
- Extend the evaluation framework with A/B testing against the current manual screening process to measure business impact (time-to-hire, offer acceptance rate).
- Deploy the dashboard as a production web application with authentication, role-based access (candidate vs HR), and database integration.
- Incorporate job market trend data (e.g., in-demand skills, salary benchmarks) to weight recommendations toward high-demand roles.

12. Pipeline Summary

Step	Description	Key Outputs
Step 2	Synthetic dataset generation (500 candidates, 200 jobs, Sri Lankan context)	candidates.csv job_postings.csv
Step 3	Text cleaning, categorical encoding, normalisation, TF-IDF vectorisation	candidates_preprocessed.csv job_postings_preprocessed.csv TF-IDF matrices
Step 4	Content-based recommendation engine with experience, domain, and location bonuses	recommendations.csv final_score_matrix.npy
Step 5	TF-IDF vocabulary analysis, Word2Vec training, embedding comparison	word2vec_skills.model candidate/job W2V embedding .npy files
Step 6	Multi-metric similarity computation, hybrid scoring, top-5 generation	step6_recommendations.csv five score matrix .npy files

Step 7	Evaluation: Precision@K, Recall@K, NDCG@K, Hit Rate@K, MAP, per-domain analysis	step7_eval_results.json
Step 8	Streamlit + Plotly interactive dashboard with Candidate Explorer, Evaluation, Bulk views	step8_dashboard.py

13. Conclusion

This internship project successfully designed and implemented an end-to-end job recommendation system. The system covers the complete pipeline from synthetic data generation through preprocessing, content-based filtering, text feature extraction, multi-metric similarity matching, quantitative evaluation, and an interactive dashboard deployment.

The hybrid scoring model, which combines TF-IDF cosine similarity with Word2Vec semantic embeddings and structural HR signals, emerged as the best-performing approach across all evaluation metrics. The Streamlit dashboard provides a usable interface that allows HR teams to explore recommendations in real time, switch between similarity metrics, and export results for further review.

While the current implementation uses synthetic data and a proxy relevance signal, the architecture is designed to scale to real-world data with minimal changes. The modular pipeline structure means that each component (data ingestion, preprocessing, scoring, evaluation, visualisation) can be upgraded independently as real data and user feedback become available.

The project demonstrates the practical value of applying NLP and machine learning techniques to the recruitment domain, with a clear pathway to production deployment and measurable impact on HR efficiency and candidate experience in recruitment industry.

14. Tools & Technologies

Tool / Library	Version	Purpose
Python	3.10+	Primary programming language
pandas	Latest	Data manipulation and CSV I/O
numpy	Latest	Matrix operations and similarity computation

scikit-learn	Latest	TF-IDF vectorisation, cosine similarity, encoders
gensim	Latest	Word2Vec training and skill embeddings
matplotlib / seaborn	Latest	Static visualisations in notebooks
Faker	Latest	Synthetic Sri Lankan candidate and job data generation
Streamlit	Latest	Interactive dashboard web application
Plotly	Latest	Interactive charts in the dashboard
scipy	Latest	Sparse matrix storage for TF-IDF

End of the Report.